CSC 641/841

Introduction to Computer Performance

Joseph Hui
jhui@sfsu.edu

Objective

- Discuss Performance monitoring and tuning in multi-tier environment
  - How to narrow down the problem

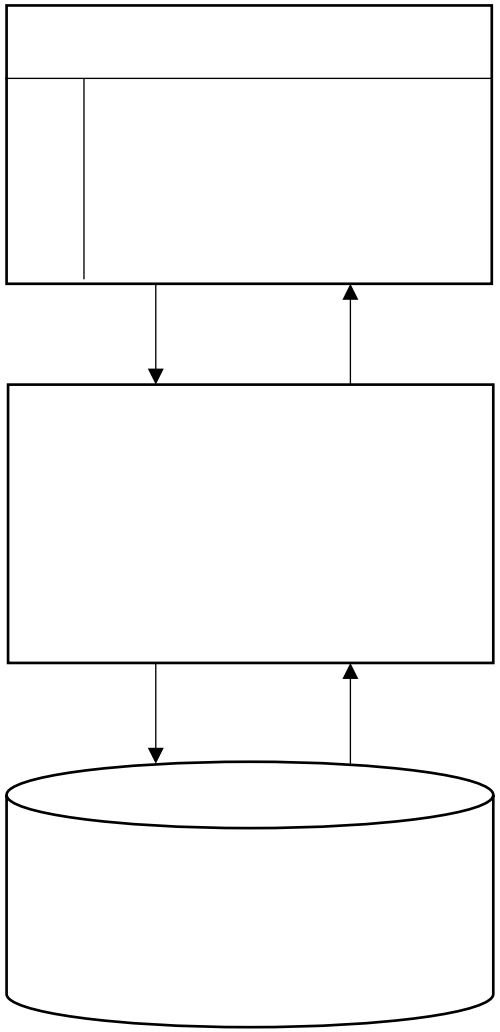
- Discuss Performance tuning in each tier

Multi-tiered Architecture

- Web Server
  - UI Server
  - HTML-Javascript

- Application Server
  - Business Logic
  - Java/C++

- Database Server
  - Data storage retrieval
  - SQL



- Apache
- MS IIS

- WebLogic
- JBoss

- Oracle Autonomous
- MS SQL Server
- MySQL

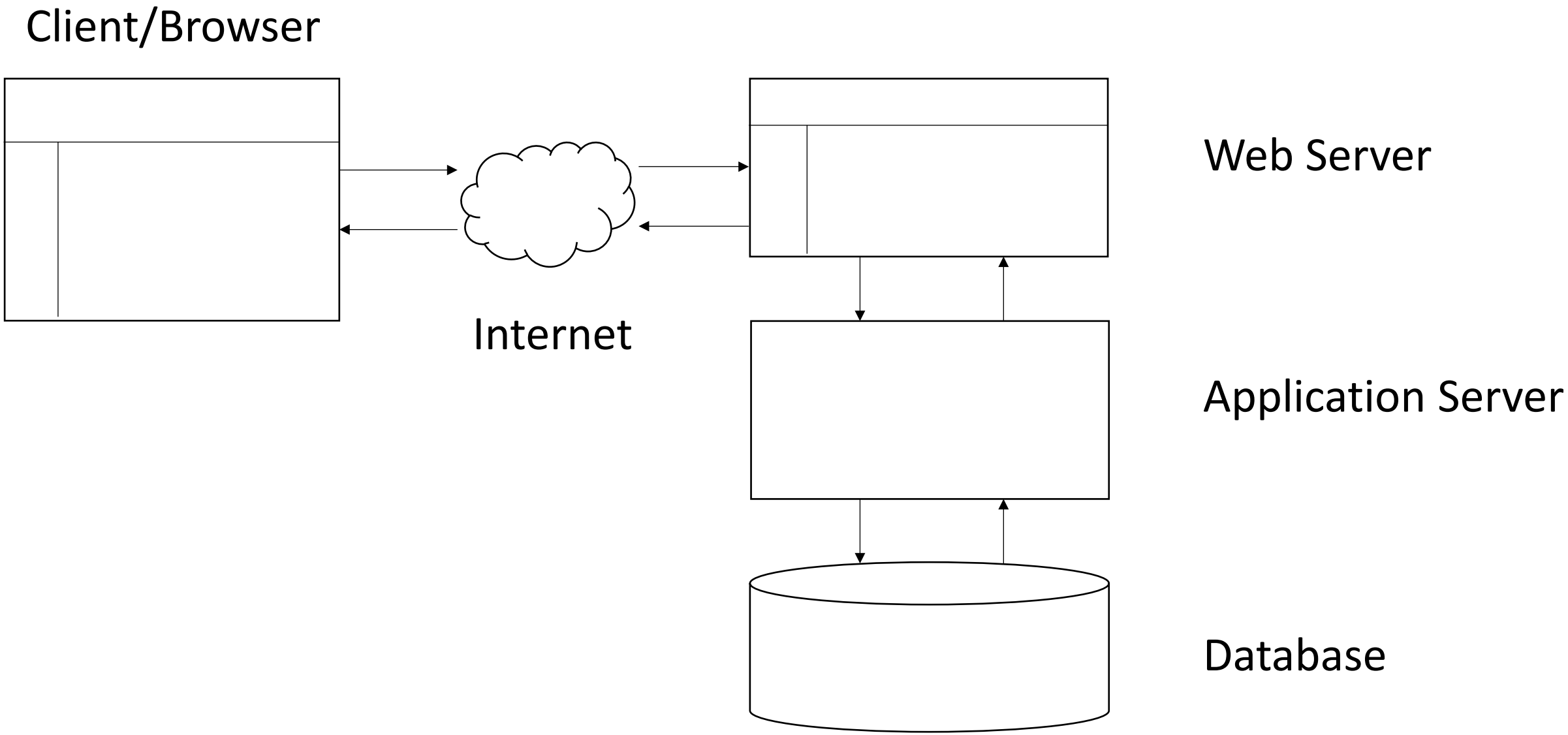
All Server-Side Machines

- Basic Machine Statistics (top, df, ntop)
  - Look for non-essential processes and kill them
    - Make sure they don’t come back on a reboot
  - Memory – verify total memory consumption is NOT close to physical memory (top)
    - Do NOT make the server swap out
  - CPU – check which process is consuming the CPU (top)
    - Should be the server software on the machine
  - Disk – verify the machine is NOT running out of disk space (df)
  - Network I/O – verify network I/O is NOT maxed out (ntop)

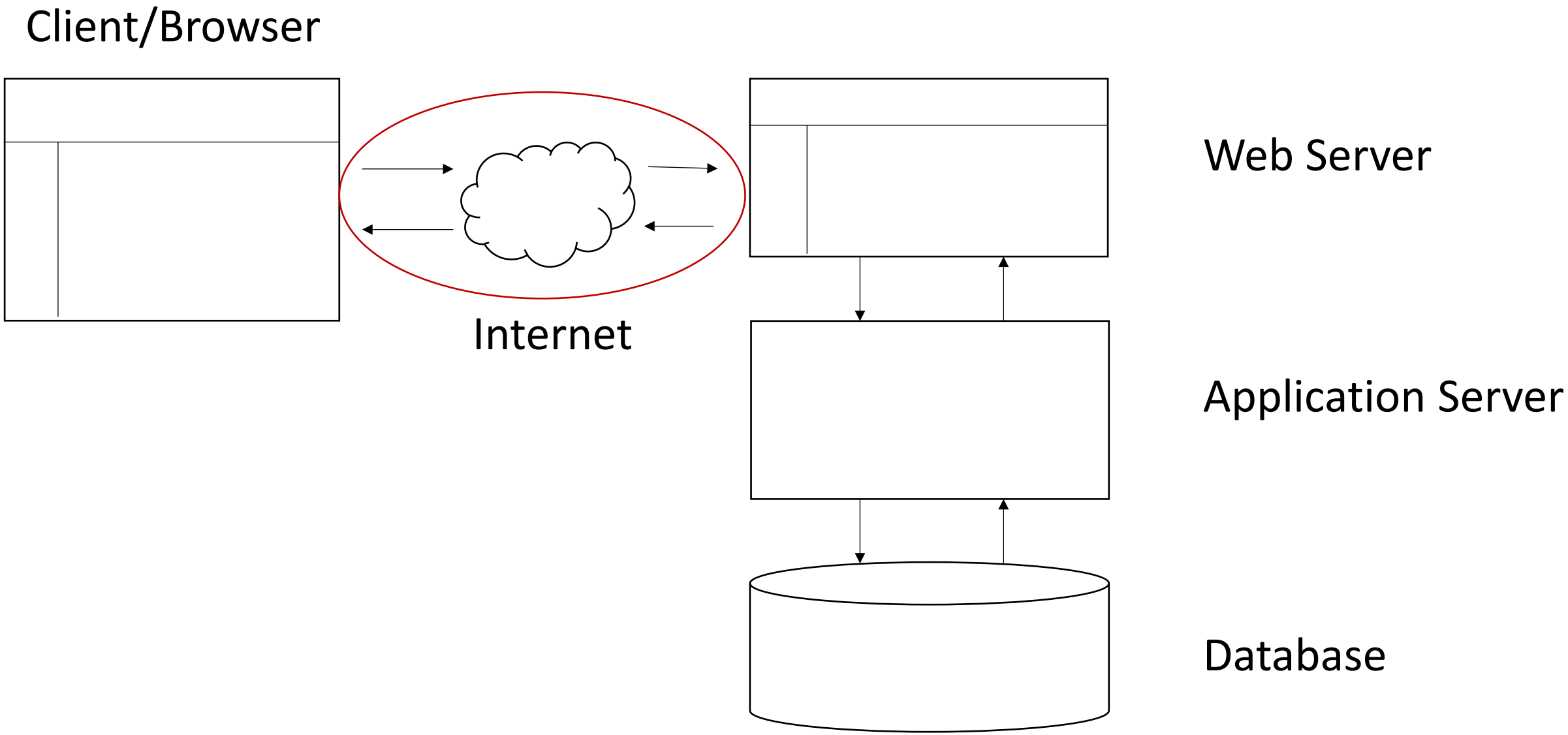
- Write a script that collects this information hourly to a file
  - Gives a historic view of machine statistics
  - Zip up older files

- Are all machines in layer overloaded or just one server?
  - For instance, if all machines in the application server layer are maxed out, scale horizontally
  - If only one machine is maxed out, need to dig deeper into the processes on that one machine

Typical Web Architecture



Typical Web Architecture



Browser to Web Server Interaction

- If all interactions with the server are slow, investigate this interaction

- Monitor the client connection and interaction with the web server
  - Time how long pages take to load on the client
  - The http access log file shows how long it took in the server to SERVE page
  - Difference in time is in network transportation

- Surprising how often customer complaints are in network transport
  - Is the client on a slow internet connection
  - Is the client in a VPN that is slowing down the connection
  - Verify connectivity speeds of client machine to internet

Stateless Interfaces

- Web Server interactions should be stateless
- Allows for horizontal scaling
- Stateful interfaces locks in user thread to a particular server machine
  - FTP does this with the current directory
  - Server machine needs to maintain state (i.e., lots of memory)
- Strive for stateless HTTP interfaces

Recognizing Patterns in interactions

- Look for patterns in the API interactions
  - Are department retrievals always followed by employee retrievals

- If patterns exist
  - Consider embedding child objects in the parent tree
  - Beware of going too deep (generally, one level is good enough)

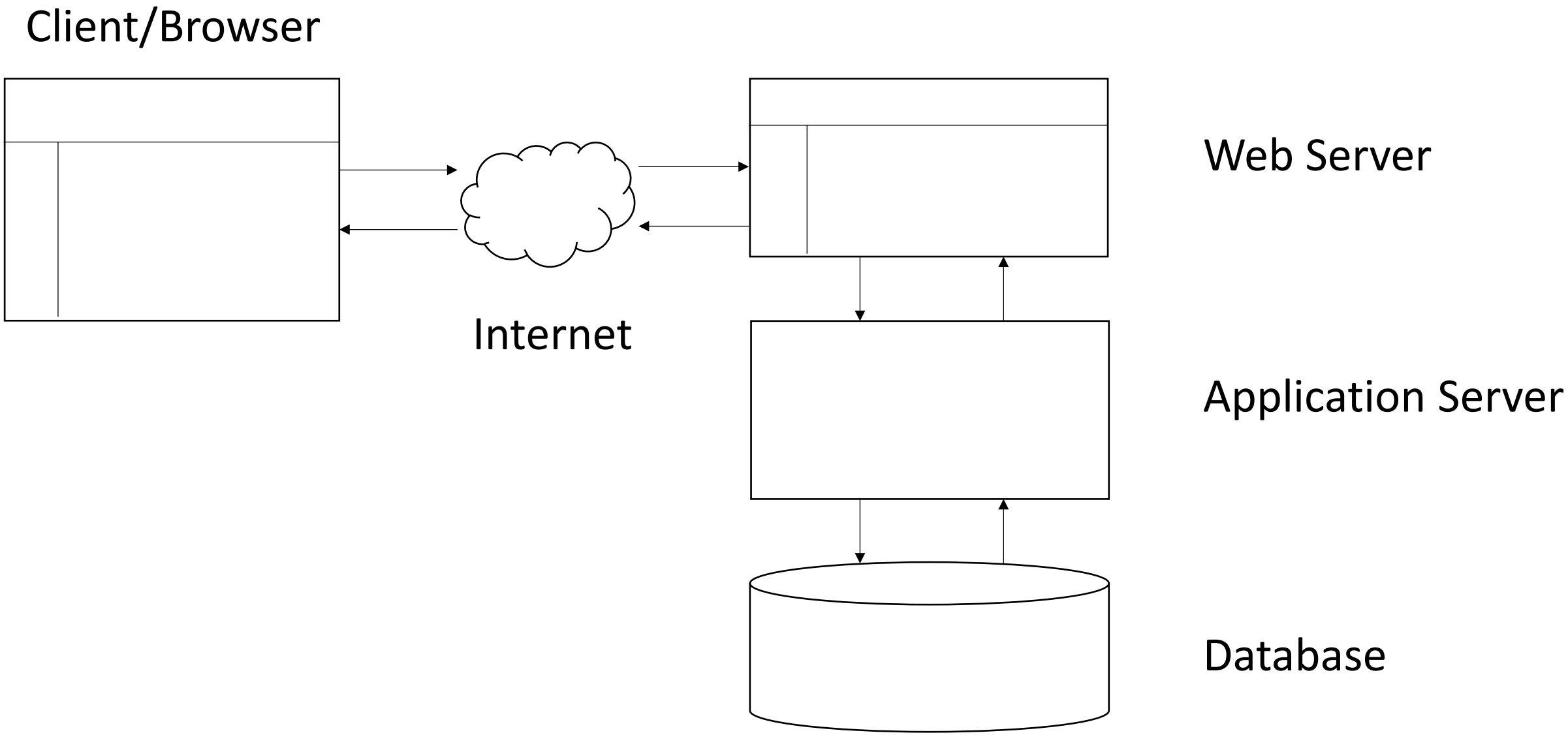
Apache Access Log

- Location of log file depends on installation and is also configurable
- Contents in the access log file are also configurable
- Configure web server to show the time spent to serve the request
  - Admin will be able to make this modification
- Compare the TOTAL difference in time between client display and web server request times
  - How many calls are there between client and server to display a page?

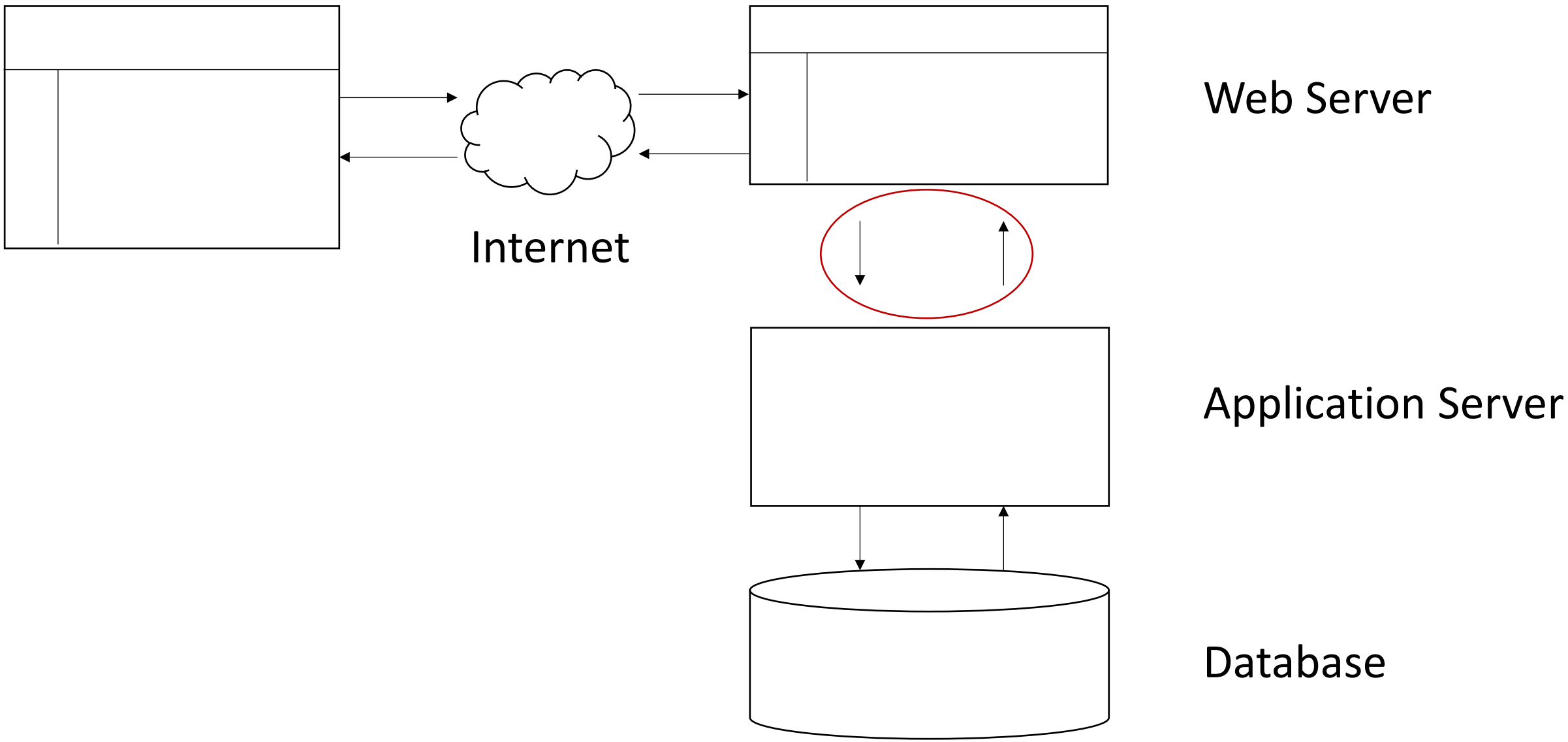
Reducing time on Web Server

- Network Cache Server optimizes interaction between client and web server
- Network Cache Server optimizes static/dynamic pages and streaming media
  - Home page gets accessed by millions
- Serving pages from Network Cache Server reduces calls to web server
  - Inktomi is one of the leaders in this area

Typical Web Architecture



Typical Web Architecture



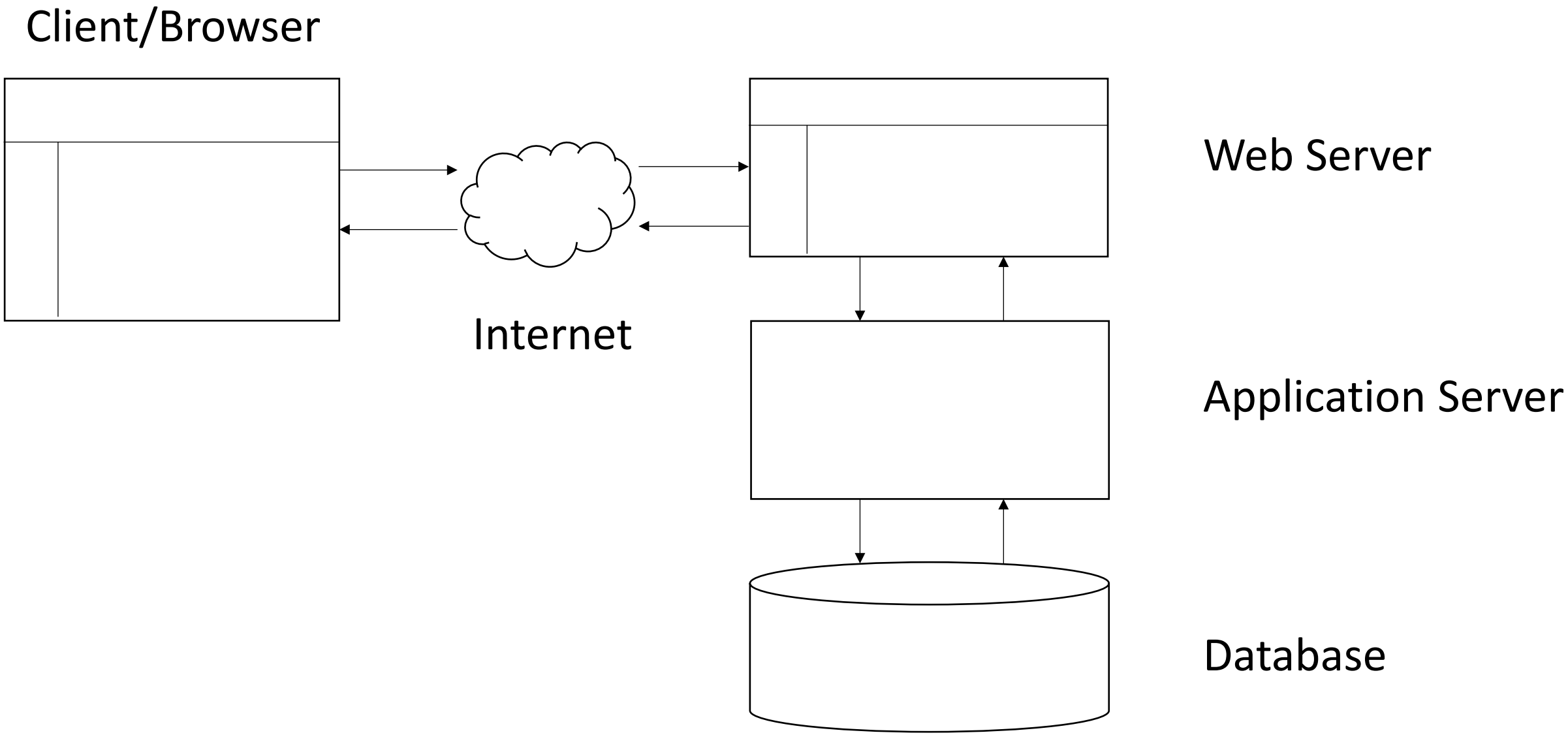
Log messages

- Log the Enterprise Context ID (ecid) when entering method
  - All log messages should be configured with ecid and timestamp
  - Log ecid, with each message, so messages can be correlated

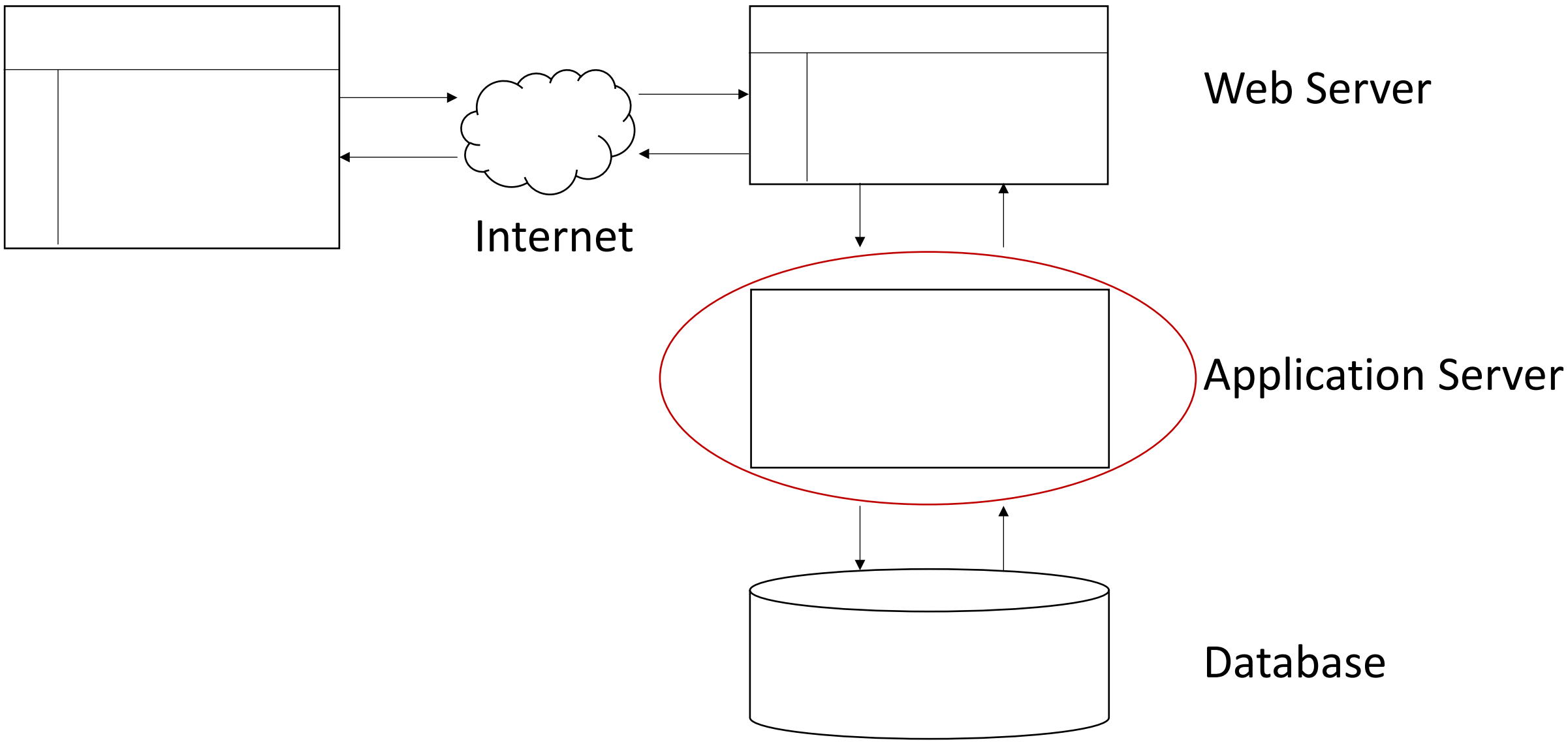
- If debug logging, log the parameters
  - Do NOT have a generic log(Level.DEBUG, “Entering with id: “ + id); which checks if DEBUG mode is set in the method.
  - Surround the log() call with a “if (Level.DEBUG) { log(Level.DEBUG, msg); }”
  - Be careful if parameters have PII data (should have standards on what can be logged)

- Log when leaving method
  - Correlating log file contents via ecid can then be used to determine execution time

Typical Web Architecture



Typical Web Architecture



Profilers

- If possible, attach a profiler to the Application Server process
  - Possible on development servers, but not production
  - Profilers give very detailed information (much more than APM)
  - UI is similar to debuggers

- Start the server with profiler flags
  - From profiler UI, attach the profiler to the server process
  - Start recording
  - Perform operation
  - Stop recording

Profiler (cont)

Hot Spots | Telemetries | Events | Tracker | **Web Services** JAX-WS client calls

Thread selection: All thread groups | Aggregation level: Methods

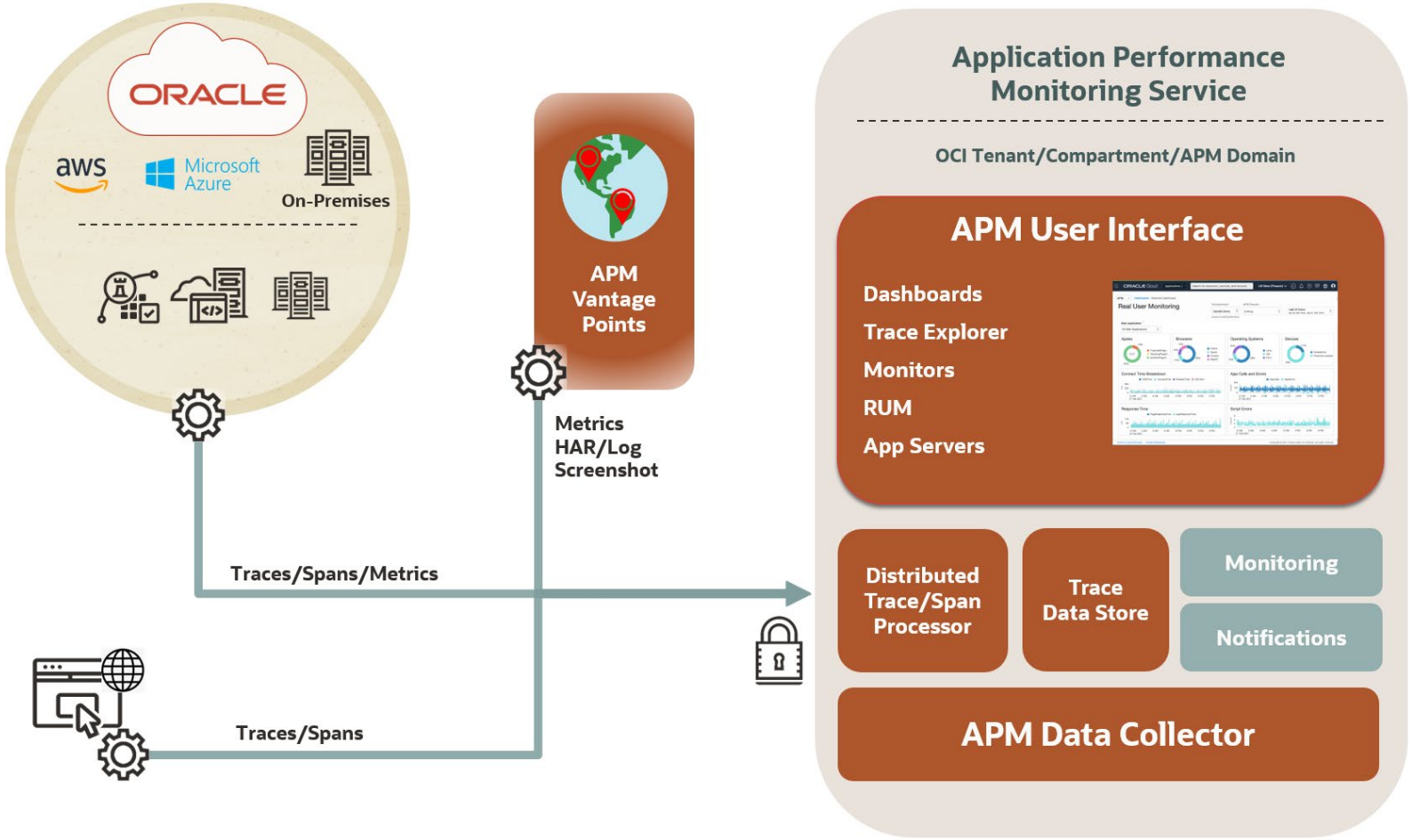
Thread status: All states

Hot spot	Self time	Average Time	Events
{http://handlers.server.demo.ejt.com/}WsHandlerService.g...	187 s (100 %)	76.195 µs	2.463
100,0% - 187 s - 2.463 hot spot inv. com.ejt.demo.server.handlers.WsHandler.getExchangeRate [com.sun.proxy.$P			
100,0% - 187 s - 2.463 hot spot inv. com.ejt.demo.server.handlers.HandlerHelper.makeWebServiceCall			
75,0% - 140 s - 1.854 hot spot inv. com.ejt.demo.server.handlers.RmiHandlerImpl.makeWebServiceCalls			
75,0% - 140 s - 1.854 hot spot inv. com.ejt.demo.server.handlers.RmiHandlerImpl.performWork			
75,0% - 140 s - 1.854 hot spot inv. com.ejt.demo.server.handlers.RmiHandlerImpl.remoteOperation			
75,0% - 140 s - 1.854 hot spot inv. RMI: 192.168.1.139			
75,0% - 140 s - 1.854 hot spot inv. java.util.concurrent.ThreadPoolExecutor$Worker.run			
18,3% - 34.361 ms - 444 hot spot inv. com.ejt.demo.server.handlers.RequestHandler.makeWebServiceCall			
18,3% - 34.361 ms - 444 hot spot inv. com.ejt.demo.server.handlers.RequestHandler.performWork			
18,3% - 34.361 ms - 444 hot spot inv. com.ejt.demo.server.handlers.RequestHandler.run			
4,4% - 8.248 ms - 108 hot spot inv. HTTP: /demo/view5			
4,4% - 8.248 ms - 108 hot spot inv. com.ejt.demo.server.DemoServer$3.run			
4,1% - 7.765 ms - 99 hot spot inv. HTTP: /demo/view4			
4,1% - 7.765 ms - 99 hot spot inv. com.ejt.demo.server.DemoServer$3.run			
3,9% - 7.284 ms - 93 hot spot inv. HTTP: /demo/view1			
3,9% - 7.284 ms - 93 hot spot inv. com.ejt.demo.server.DemoServer$3.run			
3,0% - 5.568 ms - 72 hot spot inv. HTTP: /demo/view3			
3,0% - 5.568 ms - 72 hot spot inv. com.ejt.demo.server.DemoServer$3.run			
2,9% - 5.493 ms - 72 hot spot inv. HTTP: /demo/view2			
2,9% - 5.493 ms - 72 hot spot inv. com.ejt.demo.server.DemoServer$3.run			
6,7% - 12.617 ms - 165 hot spot inv. com.ejt.demo.server.handlers.JmsHandler.makeWebServiceCall			
6,7% - 12.617 ms - 165 hot spot inv. com.ejt.demo.server.handlers.JmsHandler.handleMessage			
6,7% - 12.617 ms - 165 hot spot inv. com.ejt.demo.server.handlers.JmsHandler.onMessage			
6,7% - 12.617 ms - 165 hot spot inv. com.ejt.demo.server.DemoServer$3.run			

Application Performance Monitors (APM)

- APM can run on production servers with some overhead (<5%)
  - Depends on the amount of data collected
  - Relies on injection (code is executed at every method invocation/exit)
- Filtering is available so less data is collected and sent
- Sends snapshot information to APM server
- Also contains the application server statistics (CPU, memory, etc.)

APM (cont)


ORACLE
aws
Microsoft Azure
On-Premises
APM Vantage Points
Metrics
HAR/Log
Screenshot
Traces/Spans/Metrics
Traces/Spans
Application Performance Monitoring Service
OCI Tenant/Compartment/APM Domain
APM User Interface
Dashboards
Trace Explorer
Monitors
RUM
App Servers
Distributed Trace/Span Processor
Trace Data Store
Monitoring
Notifications
APM Data Collector

Log messages

- Access to production servers is highly restricted
- Cannot attach debugger to production servers, so log messages are critical to backtrack/debug issues
- Cannot change jar files on production servers to add debug messages
  - Environments are updated on a schedule (biweekly or monthly)
  - Put “enough” debug messages to help track issues
- Have admin change the logging level for your package

Log files

- Exceptions should not be thrown as part of typical executions (expensive)
- Do NOT swallow exceptions (i.e., catch blocks that don’t log messages nor rethrow exceptions)
- Log files get cycled (configurable size limit)
  - Periodically need to check the total file size used (admin scripts)
  - Analyze logs for WARNING/ERROR messages
    - Especially for DBMS Transaction rollback
    - sed, awk, grep, etc., are heavily used
- Careful with logging DEBUG messages
  - Lots of data gets written and slows down the server
  - If turned on, make sure admin turns off once done

Sample log contents

```
2020-07-11 03:02:38.156 [[ACTIVE] ExecuteThread: '8' for queue: 'weblogic.kernel.Default (se
lf-tuning)'] [ECID: 662023d3-f5ee-4661-932d-426a1b62e9b0-0000003f] INFO  com.example.ECIDCon
verter.ECIDService - start of printECID method
2020-07-11 03:02:38.159 [[ACTIVE] ExecuteThread: '8' for queue: 'weblogic.kernel.Default (se
lf-tuning)'] [ECID: 662023d3-f5ee-4661-932d-426a1b62e9b0-0000003f] INFO  com.example.ECIDCon
verter.ECIDService - ECID is 662023d3-f5ee-4661-932d-426a1b62e9b0-0000003f
2020-07-11 03:02:38.159 [[ACTIVE] ExecuteThread: '8' for queue: 'weblogic.kernel.Default (se
lf-tuning)'] [ECID: 662023d3-f5ee-4661-932d-426a1b62e9b0-0000003f] INFO  com.example.ECIDCon
verter.ECIDService - end of printECID method
```

History Columns

- Good to have several history columns to track updates to a row
  - Username (record who made the last update)
    - Not so useful if the APPID is always recorded
    - Try to use the userId
  - Timestamp (record when last updated)
  - Version number (counter for number of updates)
    - High version numbers are indicative of a “hotspot” on the DBMS

- Many ORMs support history columns natively

Configure JFRs

- Java Flight Recorder files are critical to track down performance issues
  - JVM needs to be started with JFR enabled
  - Overhead is “reasonable” and is generally < 1%
  - Needs a commercial license

- JFR files (JFRs) are quite large (depends on configuration)
  - Generally configured to be dumped hourly
  - Create one on exit (i.e., server crashed)

- Java Mission Control (JMC) is GUI to read/analyze JFRs

JFRs

- Events
  - Duration of wait events (critical sections)
    - Caches can be hot spot if constantly updated
  - Sample event to get a snapshot (configurable intervals)
  - Lots of information available

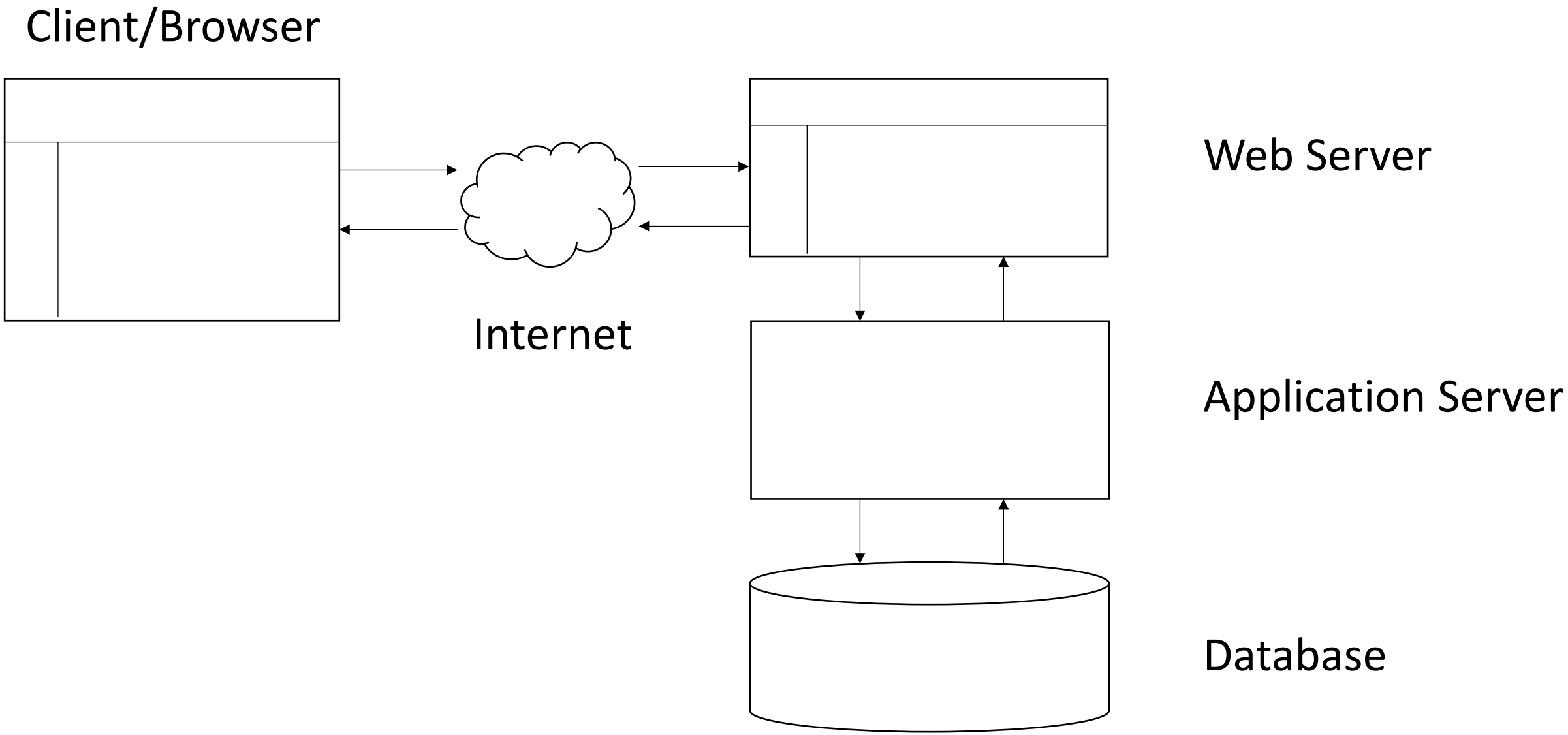
- Memory
  - Strings are the most allocated object
  - Investigating String allocations are a waste of time (just too many of them)
  - Helpful to track down where memory is used
    - Hopefully, biggest consumers are caches

- DB Connections, Threads, Memory close to being maxed out

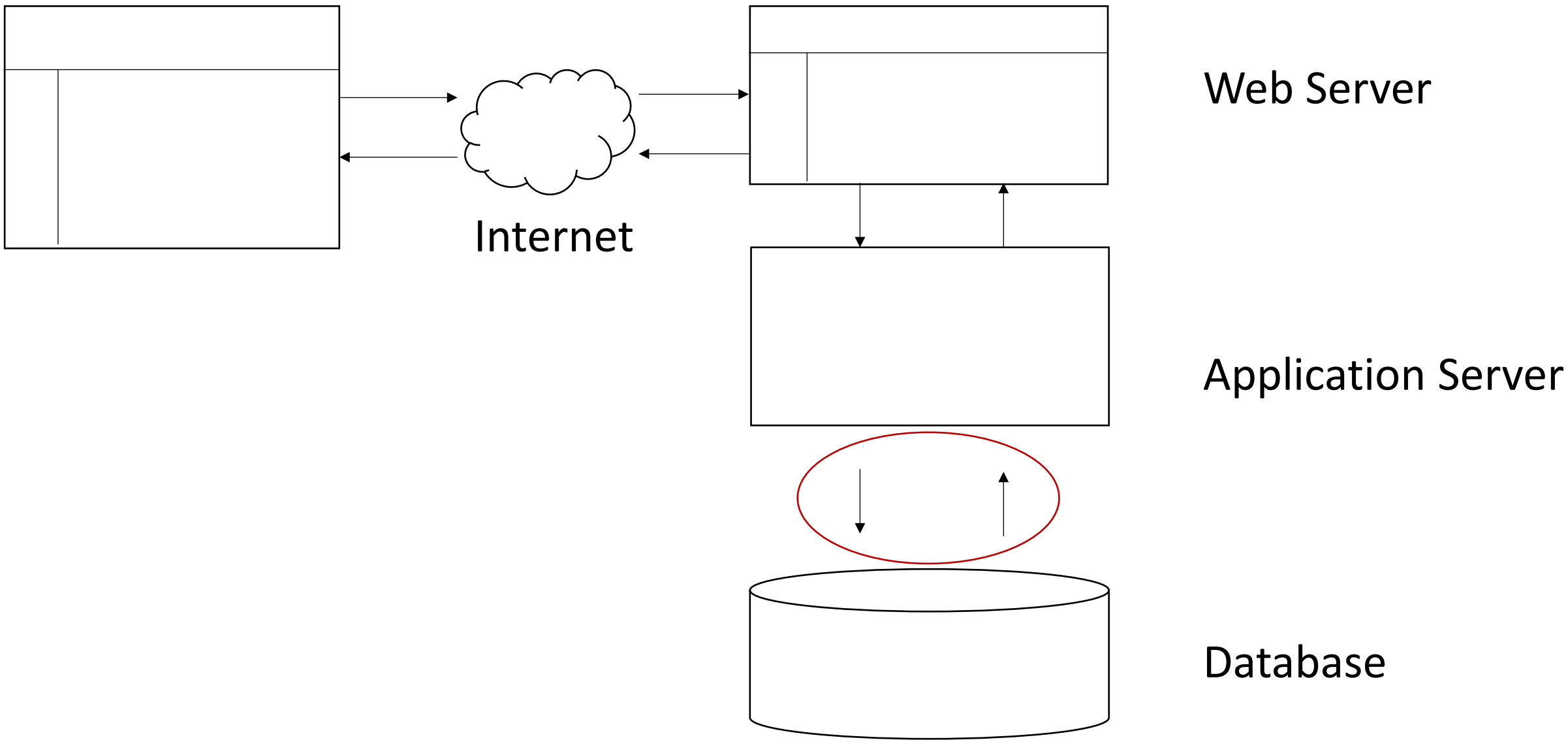
Hotspots

- Places in the code where multiple threads get funneled through
- Solution is to parallelize hot spots
- Traffic analogies are applicable to some server issues
  - Think of the Bay Bridge toll plaza
    - Many lanes to get through the hotspot
  - A bus can get many people through faster (higher throughput)
  - An individual time could be slower
- Ensure optimizing one operation does not negatively impact other operations

Typical Web Architecture



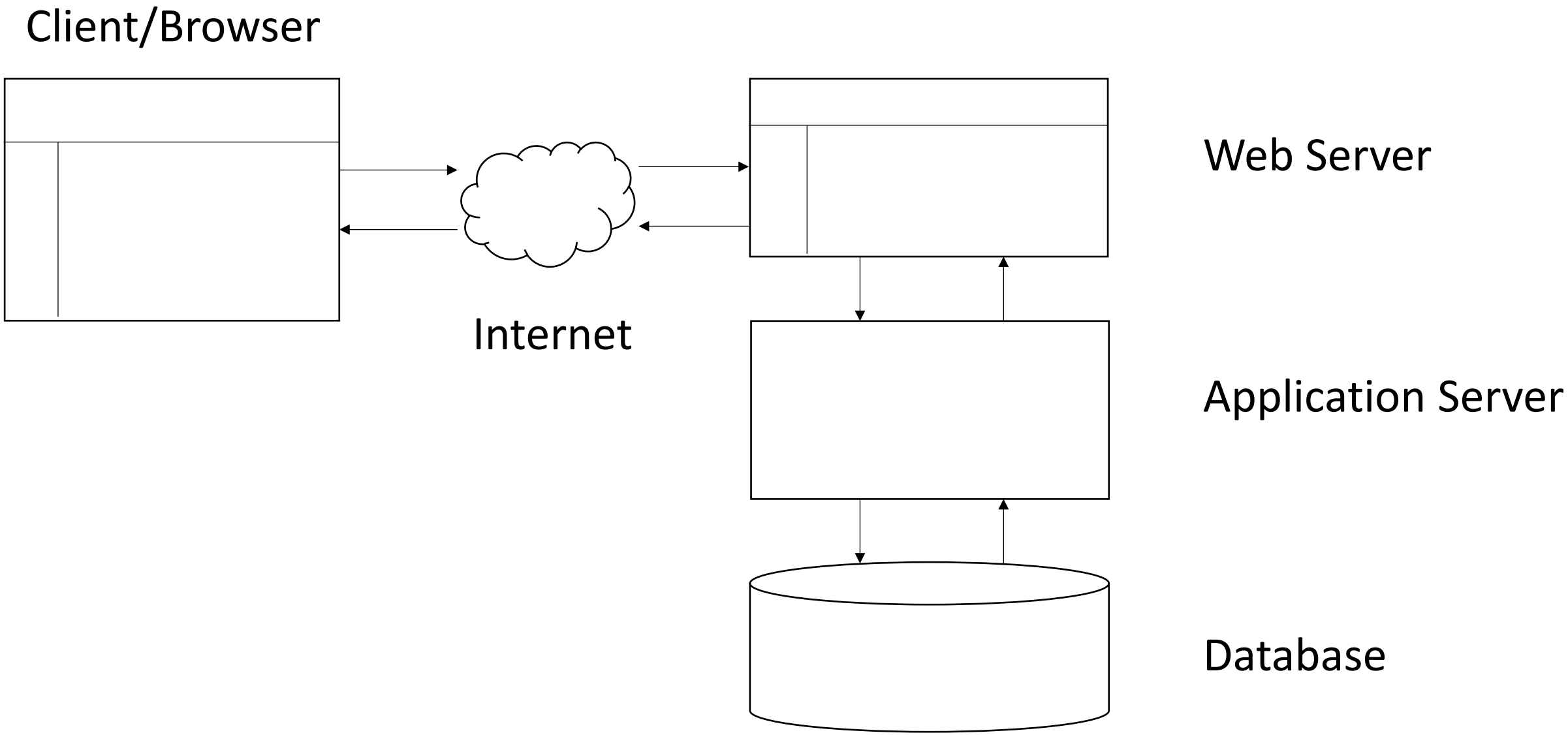
Typical Web Architecture



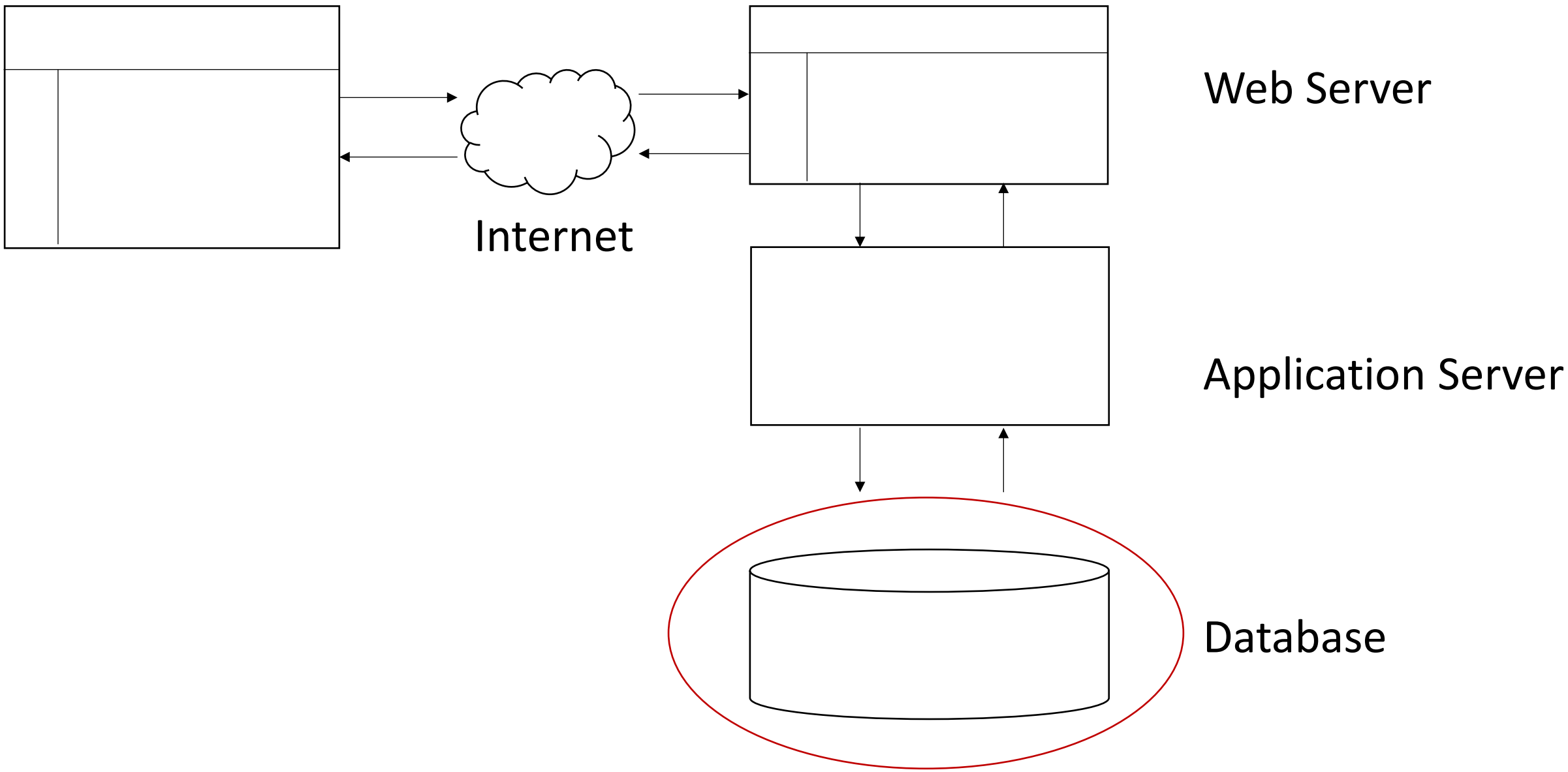
DBMS interactions

- Avoid chatty calls into the DBMS
  - Batch up SQL statements if possible
    - Instead of looping through multiple “WHERE order_id = ?” statements, use “WHERE order_id IN (?, ?, ?, ?, ?)”
  - Careful with DBMS interactions in a looping structure
    - Need to verify that there is a “reasonable” limit to number of times through a loop
    - Do NOT initialize Connection/Statement in a looping code
      - Pull any initialization out of the looping code and reuse in the loop
- Do a join to get parent/child (1 to many) relationships instead of two calls
- Put resultSet.close() and statement.close() calls into finally blocks

Typical Web Architecture



Typical Web Architecture



Automatic Workload Repository (AWR)

- AWRs are critical for DBMS performance analysis
  - Stored as html files
  - Hourly (default) and stored for 8 days
  - Can be run at anytime (need admin access)

- Top ten runtime queries (overall time, CPU, memory, I/O, etc.)

- Top ten most executed queries
  - This is where PreparedStatements are significant
  - Do NOT dynamically create statements with primary key searches

Query Optimization

- 3rd Normal Form for OLTP tables
- Denormalized for OLAP tables
- Avoid table scans
  - Any WHERE condition should have an index on the column
- Joins need to have adequate key coverage
  - Number of tables minus one conditions
- For GROUP BY queries, push all possible conditions to WHERE clause versus HAVING clause (HAVING clause should only have aggregate conditions)

Triggers and Stored Procedures

- Multiple SELECTs into a sproc
  - JDBC can detect changed ResultSet
  - Hibernate (ORM) cannot and will discard results after the first ResultSet

- Downsides
  - Sproc syntax is vendor dependent
  - Updates to sproc code requires DBA access
  - Updating customer site can get ugly